

cuttag_profiler: air-gapped CUT&Tag signal profiling with an embedded browser dashboard

Author One

Author Two

Department, Institution, City, Country

Abstract

Summary: Chromatin profiling assays such as CUT&Tag and CUT&RUN are read out by quantifying normalised coverage over large sets of genomic loci. The established tool for this step, `deeptools`, requires a two-stage pipeline and a command-line interface, and takes tens of minutes on a genome-scale locus set. Interactive alternatives are typically web services, which many clinical and industrial laboratories cannot use because aligned sequence data may not leave the premises. We present `cuttag_profiler`, a single statically linked C++20 binary that computes reference-point signal matrices directly from an indexed BAM and serves an interactive dashboard from an embedded HTTP server bound to the loopback interface. On 100,000 promoter windows it is $35.4\times$ faster than the equivalent `deeptools` pipeline while using $5.1\times$ less memory, and reproduces its meta-profile with a Pearson correlation of 0.99999. No byte of sequence data crosses a network boundary.

Availability and implementation: Source, build instructions and the benchmark harness are available at https://github.com/ORGANISATION/cuttag_profiler under the MIT licence. The binary depends only on `htslib` and the platform C library.

Contact: `author@institution.edu`

1 Introduction

CUT&Tag (Kaya-Okur *et al.*, 2019) and CUT&RUN (Skene and Henikoff, 2017) localise a histone modification or transcription factor by tethering an enzyme to a target-bound antibody. Downstream interpretation almost always reduces to the same operation: for each of N loci, compute normalised read coverage in B bins across a fixed window anchored on a reference point such as a transcription start site, then summarise the resulting $N \times B$ matrix as a meta-profile and a heatmap.

Two practical problems attach to that operation.

First, cost. The `deeptools` suite (Ramírez *et al.*, 2016) performs it in two stages: `bamCoverage` converts the BAM into a bigWig coverage track, and `computeMatrix` windows that track. Each stage rewrites the data, and the intermediate track is written at genome scale even when the loci of interest cover a small fraction of the genome.

Second, access. The scientists who interpret these matrices are usually not the scientists who can operate a command line, and the interactive tools that close that gap are predominantly hosted services. Laboratories operating under GDPR, HIPAA or an institutional data-governance policy frequently prohibit transmitting aligned human sequence data to third-party infrastructure, which disqualifies hosted viewers regardless of their merits.

`cuttag_profiler` addresses both. It computes the matrix in one pass directly from the indexed BAM, and it ships its own dashboard inside the executable, served over the loopback interface only.

2 Methods

2.1 Architecture

The tool is a single C++20 executable with no runtime dependency beyond `htslib` and the platform C library; `htslib` is linked statically, so deployment is a file copy. It has two modes: a headless `profile` subcommand for pipelines, and a `serve` subcommand that starts the dashboard.

2.2 Parallel indexed query

Work is distributed one locus per task over a pool of worker threads, block partitioned so that each worker writes a contiguous stripe of the output matrix. `htslib`'s index and header are immutable once loaded and are shared; its `htsFile` handles carry mutable decompression state and are not, so each worker leases a private handle from a pool.

A design decision worth recording is what *not* to share. `htslib` offers a thread pool that parallelises BGZF block inflation, and the intuitive configuration is one pool sized to the machine. For this access pattern that is actively harmful: with per-region parallelism already saturating the cores, routing every block through a shared pool serialises inflation behind the pool's queue and adds a hand-off per block. Measured on the benchmark below, a 12-thread shared pool cost 5.95s wall time against 1.38s with no pool at all, and consumed 52s of CPU against 6.4s. The default is therefore no shared pool: each handle inflates on the thread that owns it. A pool is created only when the caller explicitly requests single-threaded operation, where it can still overlap inflation within one query.

Per-thread reduction buffers are allocated on 64 byte boundaries and padded to whole cache lines, so no two workers share a line while accumulating column sums.

2.3 Signal quantification

For a locus with anchor a , the window $[a - u, a + d)$ is divided into $B = \lceil (u + d)/w \rceil$ bins of width w . For minus-strand features the window is mirrored and the resulting row reversed, so that bin index always runs $5' \rightarrow 3'$.

Each alignment contributes to the bins its reference span overlaps. For properly paired data the span is the full sequenced fragment, inferred from `TLEN` and counted once through the leftmost mate; this is the correct footprint for CUT&Tag, where the fragment marks the protected region. Because a fragment may begin before the window and still intersect it, the underlying iterator query is widened to the left by the maximum fragment length; omitting this silently truncates signal at window edges.

Two tallies are accumulated per bin: the number of overlapping fragments, and the number of aligned bases, the latter honouring the CIGAR string so that deletions and reference skips do not inflate coverage. Reported signal is either fragments per bin (the default, matching `deeptools`) or mean per-base depth. Normalisation is one of raw, CPM, RPKM or BPM; the library size used as the denominator is read from the index metadata in $O(\text{contigs})$ rather than by streaming the file.

2.4 Embedded dashboard

`serve` starts an HTTP server (`cpp-httpplib`) bound to 127.0.0.1 and opens the user's browser. Three properties enforce the air gap:

1. the listener refuses to bind a non-loopback address unless the operator passes an explicit override flag, which also prints a warning;
2. every asset is served from a byte array compiled into the binary by a CMake generator step, so no CDN, font host or analytics endpoint is contacted;
3. responses carry `Content-Security-Policy: default-src 'self'`, so even a modified asset has no route out.

The dashboard posts a JSON query description to `/api/profile` and receives the meta-profile, a band-averaged heatmap and run statistics. Full matrices are retrieved separately as TSV, keyed by a run token, rather than serialised into every interactive response.

3 Results

3.1 Benchmark design

We compared against `deeptools computeMatrix reference-point`. Because `computeMatrix` consumes a bigWig rather than a BAM, the comparable unit of work is the two-stage pipeline an analyst actually runs: `bamCoverage` followed by `computeMatrix`. Both stages are timed, and the speedup is quoted against their sum; the `computeMatrix` stage is also reported alone, which is the appropriate comparison for a reader who already holds a cached coverage track.

The dataset is a synthetic paired-end CUT&Tag library of 4 million fragments (8 million alignments, 87 MB BAM) over a 250 Mb chromosome, with 100,000 annotated promoters, of which 55% of fragments are drawn from a TSS-centred peak ($\sigma = 180\text{ bp}$) and the remainder from uniform background. It is generated by `scripts/make_demo_data.py` at a fixed seed, so the benchmark is reproducible without redistributing controlled-access data. Both tools were given 12 threads and $\pm 2\text{ kb}$ windows in 50 bp bins, three replicates each, on an Apple M-series workstation.

3.2 Runtime and memory

Table 1: Wall-clock time and peak resident memory over 100,000 promoter windows, 12 threads, mean of three replicates.

Stage	Time (s)	Peak RSS (MB)
<code>cuttag_profiler</code> (BAM $\rightarrow$ matrix)	1.42	84.7
<code>deeptools bamCoverage</code> (BAM $\rightarrow$ bigWig)	7.11	320.7
<code>deeptools computeMatrix</code> (bigWig $\rightarrow$ matrix)	43.06	432.4
<i>deeptools pipeline total</i>	<i>50.17</i>	<i>432.4</i>

Table 1 gives $35.4\times$ end-to-end and $30.4\times$ against `computeMatrix` alone, at $5.1\times$ lower peak memory. The memory difference follows from the architecture: no genome-wide intermediate track is materialised, so resident memory scales with the requested matrix rather than with the genome.

3.3 Concordance with deeptools

Agreement was assessed on the same matrices (`scripts/compare_matrices.py`). The two tools normalise against different denominators — `deeptools` scales binned coverage derived from a bigWig, `cuttag_profiler` scales fragment counts taken directly from the BAM — so the tools are interchangeable if and only if the signal *shape* matches up to one global scale factor, which is what we test.

The meta-profiles correlate at $r = 0.99999$, with a maximum residual of 0.38% of peak signal after fitting a single global scale factor. Per-locus agreement, over a systematic sample of matched rows, has median $r = 0.988$, and 99.8% of loci exceed $r = 0.95$. Residual disagreement is attributable to the intermediate binning `deeptools` performs when building the bigWig, which the direct query avoids.

4 Discussion

The speedup has a structural rather than a micro-optimisation explanation. `deeptools` materialises a genome-wide coverage track and then windows it, so its cost scales with genome size; `cuttag_profiler` queries only the intervals requested, so its cost scales with the loci of interest. Whenever those loci cover a modest fraction of the genome — the usual case for promoter, enhancer or peak-set analyses — the direct query does proportionally less work. The gap narrows when the

locus set approaches whole genome coverage, and a cached bigWig that is reused across many locus sets amortises the conversion, which is why Table 1 reports the stages separately.

Two limitations should be stated. The benchmark uses simulated data, chosen so the comparison is reproducible without redistributing controlled-access alignments; the concordance analysis, not an absolute enrichment value, is what the simulation is asked to support. Second, BPM normalisation is computed over the supplied locus set rather than genome-wide, since no genome-wide pass is performed; where a genome-wide BPM denominator is required, CPM or RPKM should be used instead.

The design generalises past CUT&Tag: nothing in the query or scoring layer is assay-specific, and the same binary profiles ChIP-seq or ATAC-seq alignments. The air-gap property is the part we would emphasise for adoption. A tool that is fast but requires a data-transfer agreement is, in a regulated laboratory, not usable at all; one that runs entirely on the analyst’s own workstation needs no such agreement.

Acknowledgements

We thank the `htslib` and `deeptools` developers, whose work this tool builds on and measures itself against.

Funding

This work was supported by [funder, grant number].

References

- Kaya-Okur, H.S. *et al.* (2019) CUT&Tag for efficient epigenomic profiling of small samples and single cells. *Nat. Commun.*, **10**, 1930.
- Skene, P.J. and Henikoff, S. (2017) An efficient targeted nuclease strategy for high-resolution mapping of DNA binding sites. *eLife*, **6**, e21856.
- Ramírez, F. *et al.* (2016) deepTools2: a next generation web server for deep-sequencing data analysis. *Nucleic Acids Res.*, **44**, W160–W165.
- Bonfield, J.K. *et al.* (2021) HTSlib: C library for reading/writing high-throughput sequencing data. *GigaScience*, **10**, giab007.
- Li, H. *et al.* (2009) The Sequence Alignment/Map format and SAMtools. *Bioinformatics*, **25**, 2078–2079.